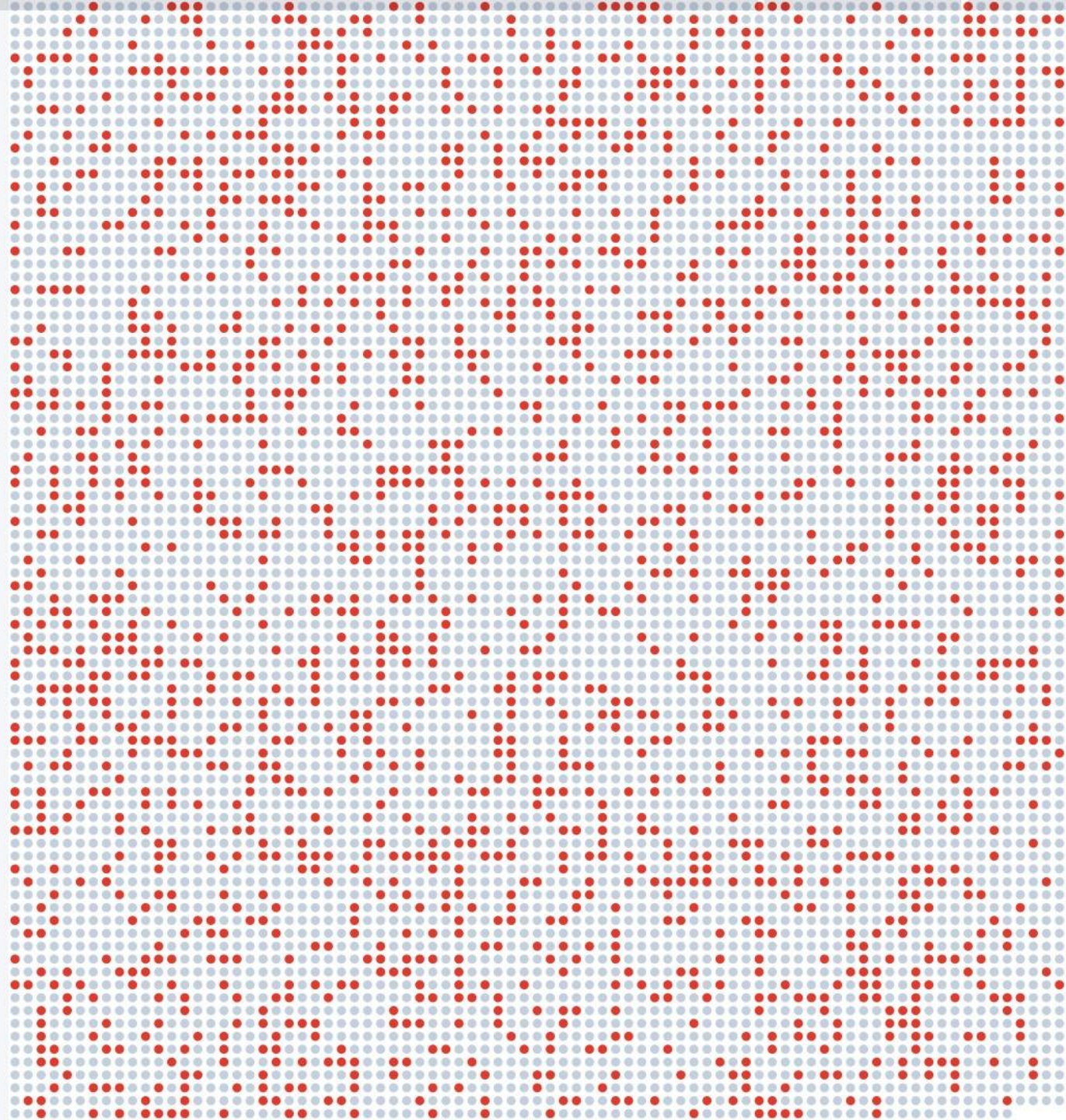


SubscribelQ

Find the customers who are about to leave, and decide who is worth saving.

● 1,869 customers churned

● 5,174 stayed



**Churners took
\$139k of monthly
revenue with
them.**

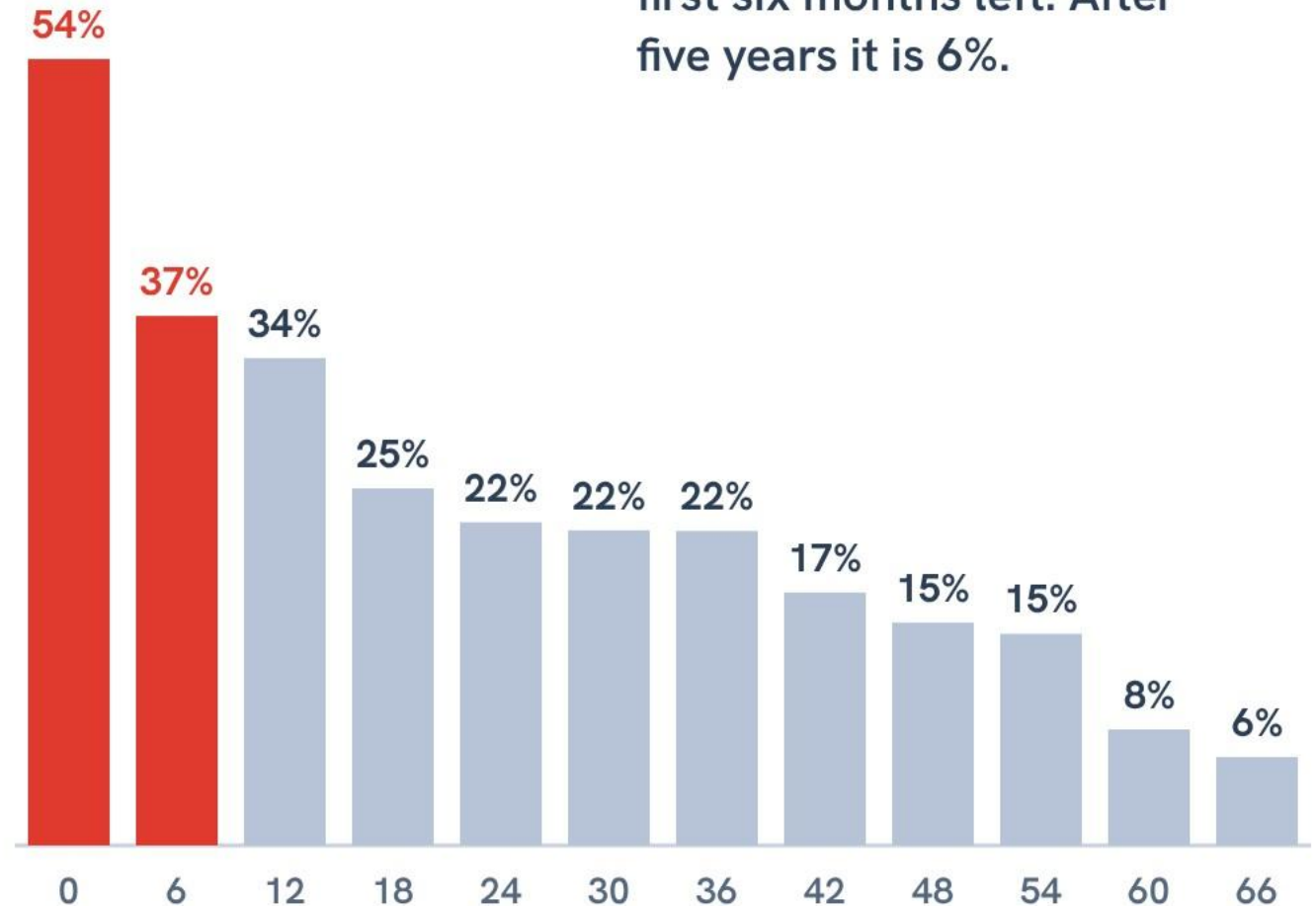
26.5% of customers
churned,
1,869 of 7,043

55% of those left in their
first twelve months

Churn rate by months as a customer

Customers grouped in six-month bands

54% of customers in their
first six months left. After
five years it is 6%.



Months as a customer, start of each band

Who it is for

Retention manager

Owns

The monthly churn number

Works from

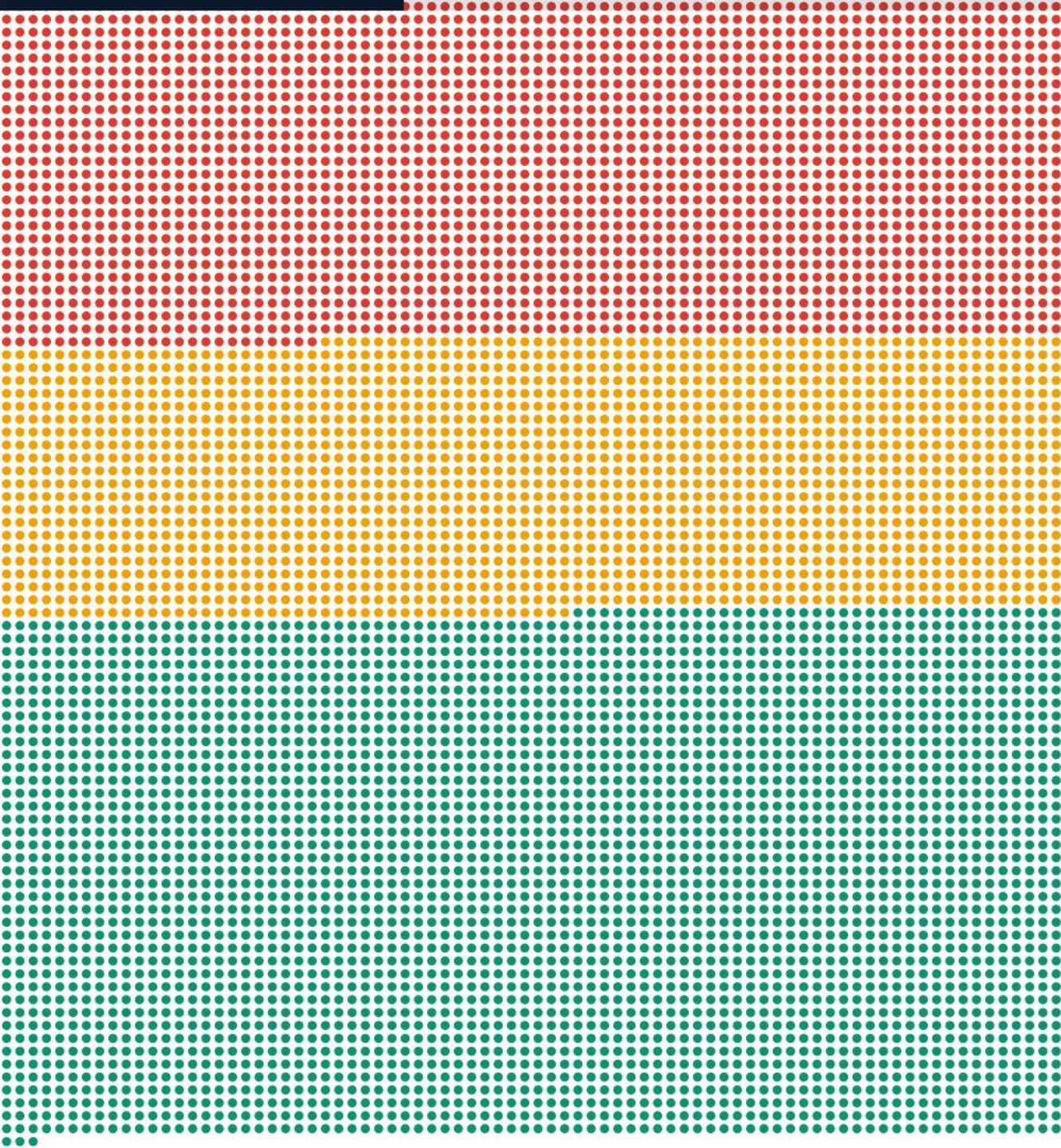
A spreadsheet and a weekly meeting

Needs

A short list, a reason, and a price on each save

From a guess to a ranked list.

	Today	With SubscribelQ
Who is at risk	Unknown until they cancel	2,104 customers, ranked
Why	Nobody can say	Reasons shown for each customer
What to offer	One discount for everyone	A play for each of four segments
What a save is worth	A guess	Monthly revenue and lifetime value on every account



The model sorts every customer into a risk tier.

A year of revenue sits in the high-risk tier

\$1.96M

Risk tier	Customers	Revenue a month	Actually churned
● High	2,104	\$163k	59%
● Medium	1,699	\$119k	26%
● Low	3,240	\$174k	6%

Churned is cross-validated: each customer was scored by a model that never saw them.

Contact the riskiest
30% and you reach
two thirds of the
churners.

2.8x

back on every dollar of offers.
Contacting everyone returns 1.2x.

Our assumptions, adjustable

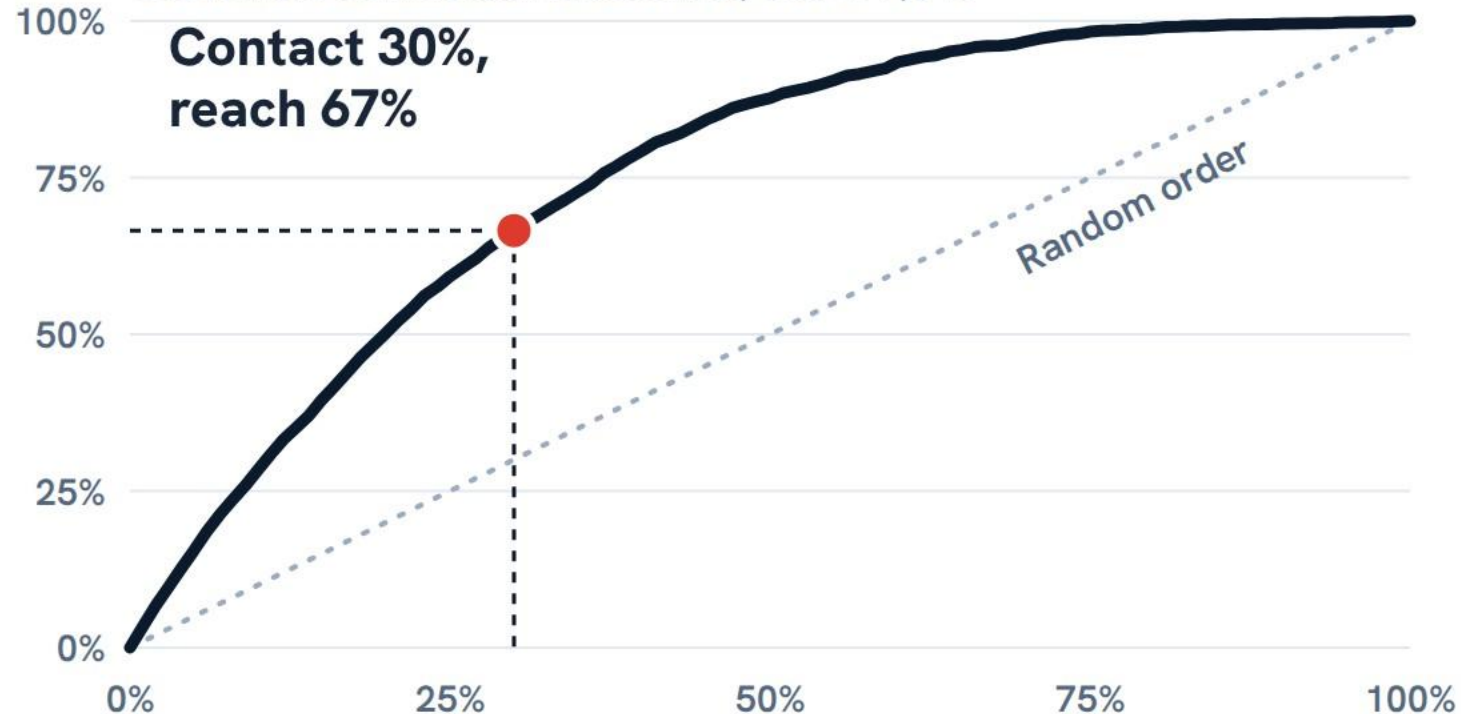
\$40 per offer made

20% of contacted churners saved

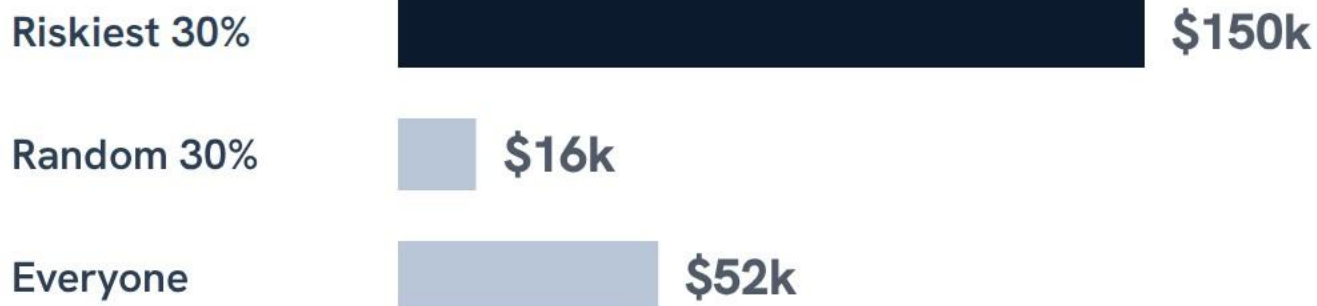
12 months of their bill kept

Share of all churners reached

Customers contacted riskiest first, out of 7,043



Net return a year, after offer costs



Contract type is the clearest churn signal.

Churn rate by contract type

Month-to-month, 3,875 customers



One year, 1,473 customers



Two year, 1,695 customers



Correlation, not proof. An A/B test will show how much a longer contract changes behavior.

Recommended play by segment

High-spend month-to-month

2,271 customers | \$81 a month | 48% churned

Offer a 12-month price lock.

At-risk new customer

1,705 customers | \$32 a month | 25% churned

Add free security and tech support for 3 months.

High-value loyal

1,905 customers | \$93 a month | 15% churned

Personal call from an account manager.

Long-tenure, low spend

1,162 customers | \$35 a month | 5% churned

Leave alone. Revisit for an upsell.

Live demonstration

Now we score a real customer.

1 Read the headline number and the risk tiers

2 Work the call list from the top

3 Open one customer and read why



Deploy

Who to call Why they leave Explore Customer

Know which subscribers are about to leave, and what it costs you.

SubscribeIQ scores every customer for churn risk, groups them into segments, and estimates what each one is worth. It turns a raw customer list into a ranked set of people worth saving this week.

Monthly revenue in high-risk accounts

\$163,221

2,104 customers (30% of the base) are more likely than not to churn.

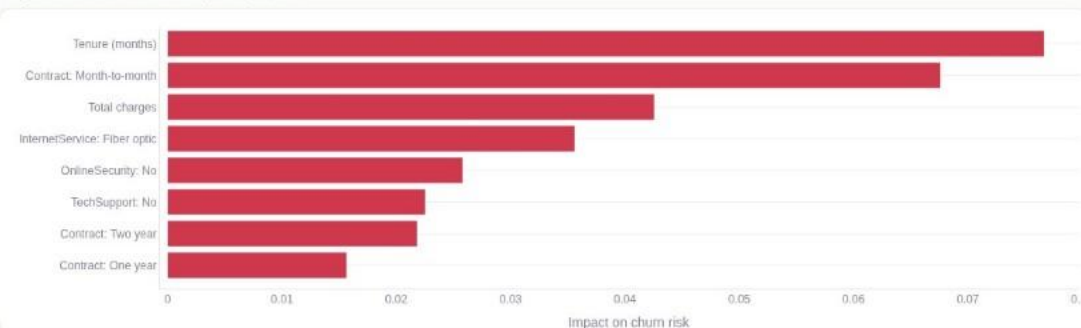
Who to call first

High-risk customers ranked by what they pay each month, so the biggest saves come first. Start at the top.

Customer 2265 High-spend month-to-month	Churn risk 75%	Monthly \$113	Est. LTV \$151	Act now
Customer 1568 High-spend month-to-month	Churn risk 73%	Monthly \$111	Est. LTV \$152	Act now
Customer 818 High-spend month-to-month	Churn risk 70%	Monthly \$111	Est. LTV \$160	Act now
Customer 574 High-spend month-to-month	Churn risk 63%	Monthly \$111	Est. LTV \$176	Act now

Why this score

Each bar is one factor pushing this customer's churn risk up (rose) or down (green). Longer bars matter more for this specific person.



Train once

python src/model.py, or automatically on the first run



Score every upload

app.py, on every upload and on first load



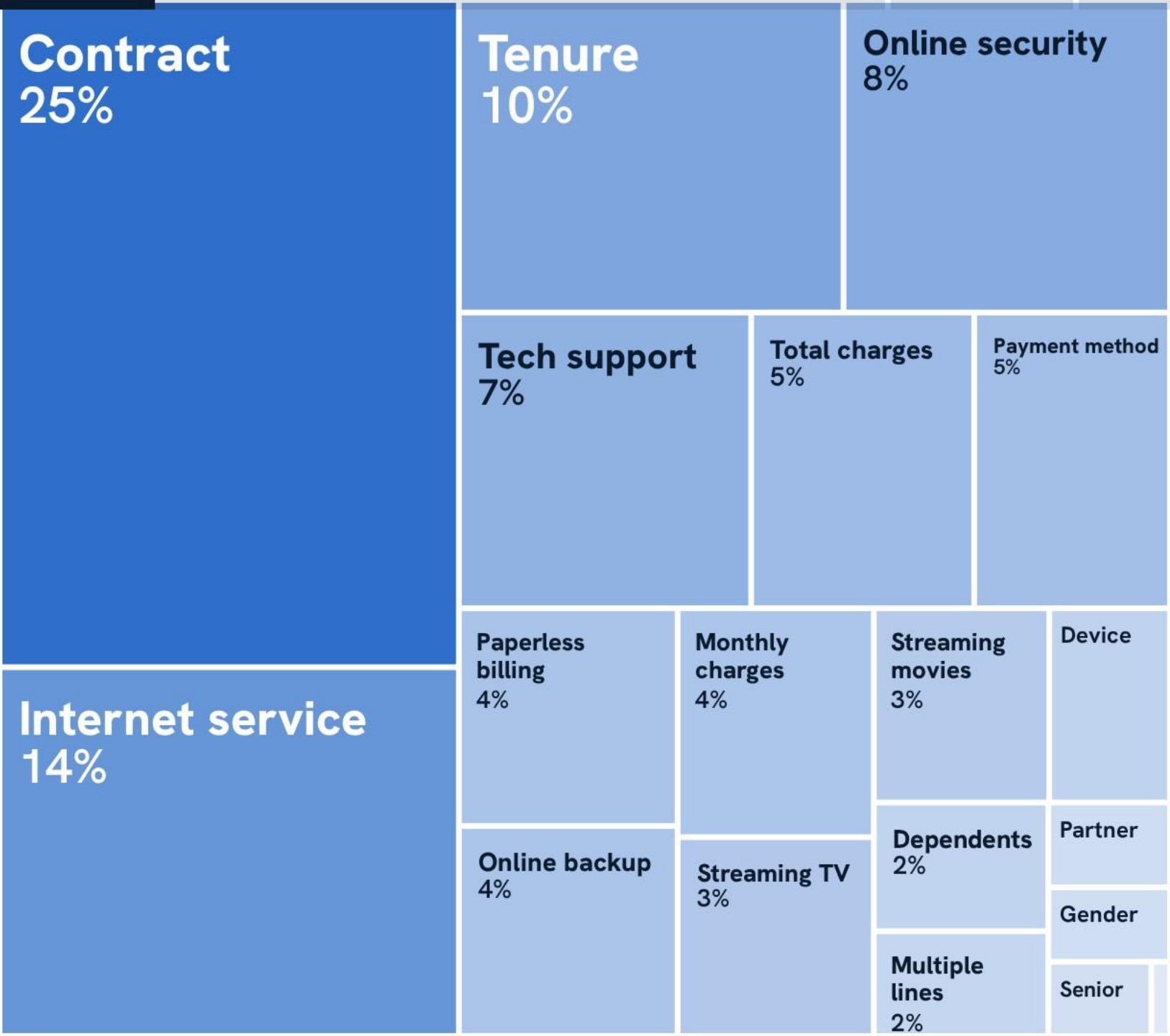
We kept every field and let SHAP rank them.

Inputs
19 fields: 3 numbers and 16 categories

Dropped
customerID, an identifier and not a signal

RFM, adapted
Tenure is recency, charges are monetary. Billing is monthly, so frequency is constant.

Tile size is each field's average effect on a score (SHAP, 160 customers).



Scale the numbers so none of them dominates.

K-Means measures distance, so TotalCharges would drown out tenure.

Before scaling, on one shared axis

tenure: 0 to 72

MonthlyCharges: 18 to 119

TotalCharges: 0 to 8,685

After StandardScaler, mean 0 and spread 1

tenure: -1.3 to 1.6

MonthlyCharges: -1.5 to 1.8

TotalCharges: -1.0 to 2.8

Turn categories into 0s and 1s.

An unseen category scores as all zeros instead of crashing the app.

Contract	Month-to-month	One year	Two year
Month-to-month	1		
Two year			1
One year		1	
Month-to-month	1		
Two year			1
Three year new in an upload			

All zeros

The forest and the baseline tie on AUC.

Model	AUC	Precision	Recall
Logistic regression	0.842	0.50	0.78
Random forest	0.842	0.53	0.77

Why we kept the forest

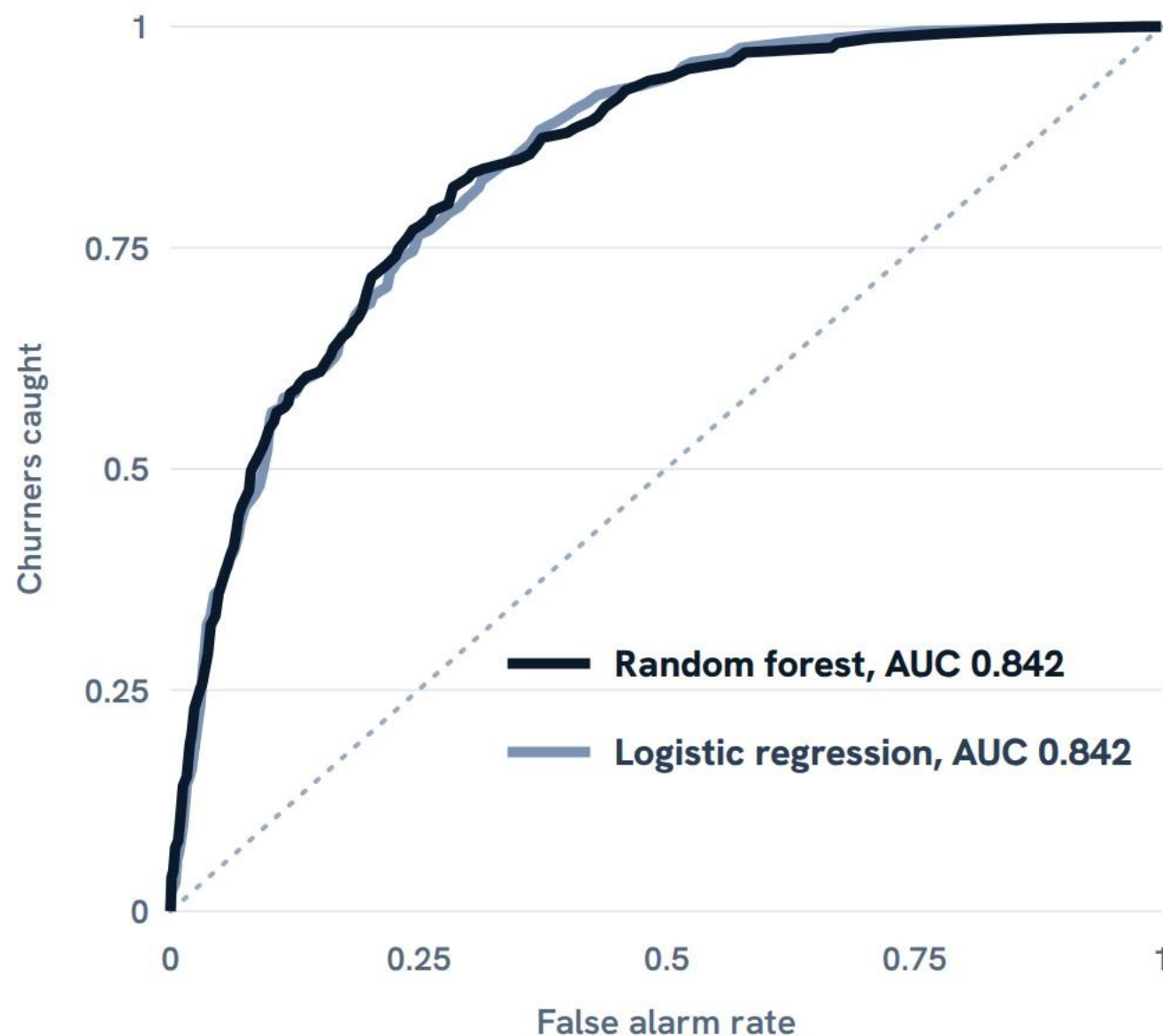
Fewer false alarms, and exact SHAP explanations for trees.

Class imbalance

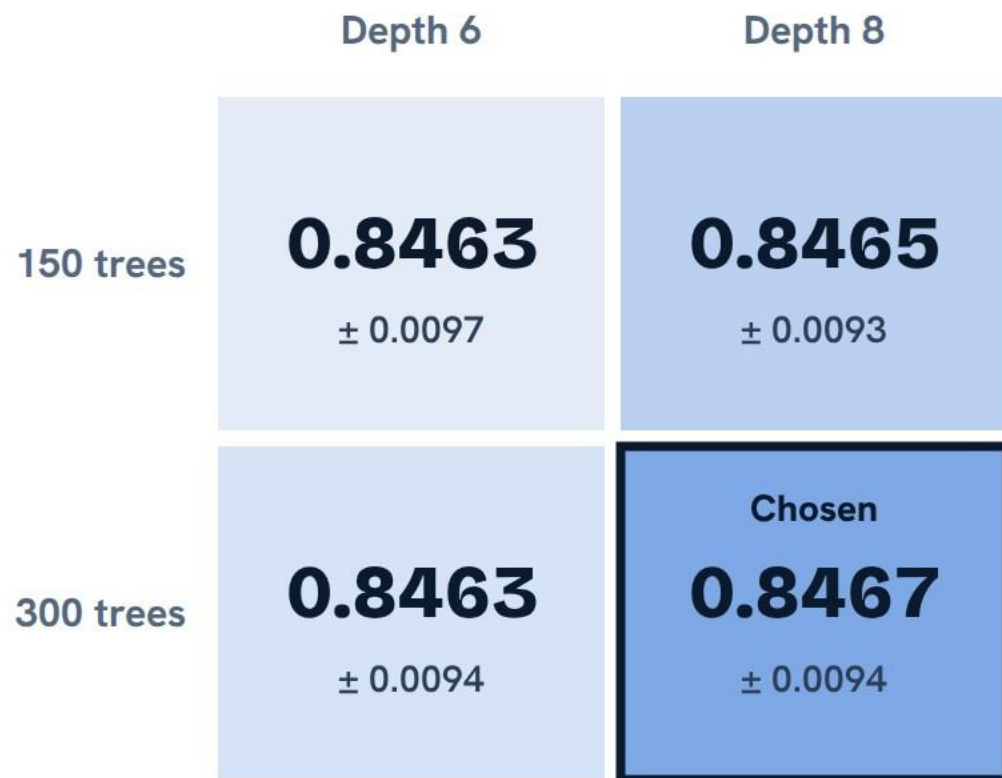
Balanced weights: a churner counts 1.9 times a stayer, with no synthetic oversampling.

ROC curve on the held-out test set

1,409 customers the models never trained on

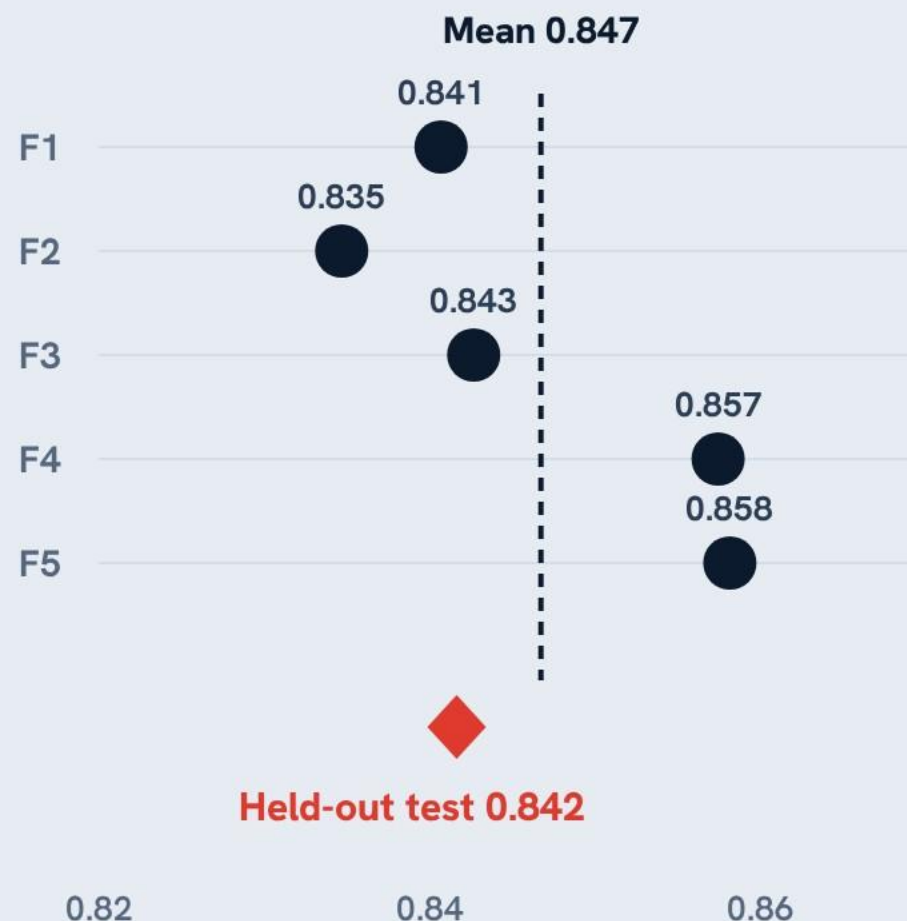


**Tuning barely moves the score.
The data sets the ceiling.**



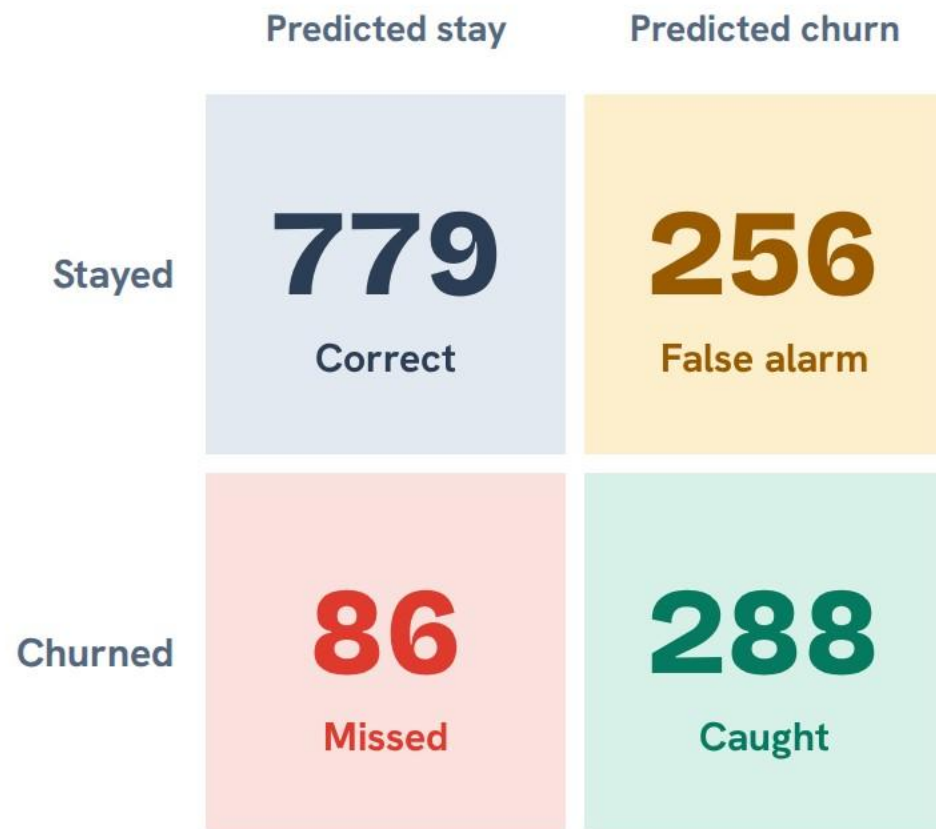
Mean ROC-AUC across 5 stratified folds, training data only.

Five folds, one answer



**Settings differ by 0.0005.
Folds differ by 0.023.**

Held-out test set, 1,409 customers



Recall 0.77: we catch 288 of 374 churners.

Precision 0.53: about half of those flagged would have stayed.

It finds month-to-month churners and misses almost everyone else.

Share of churners the model catches

Contract

Month-to-month 86%

One year 3%

Two year 0%

Months as a customer

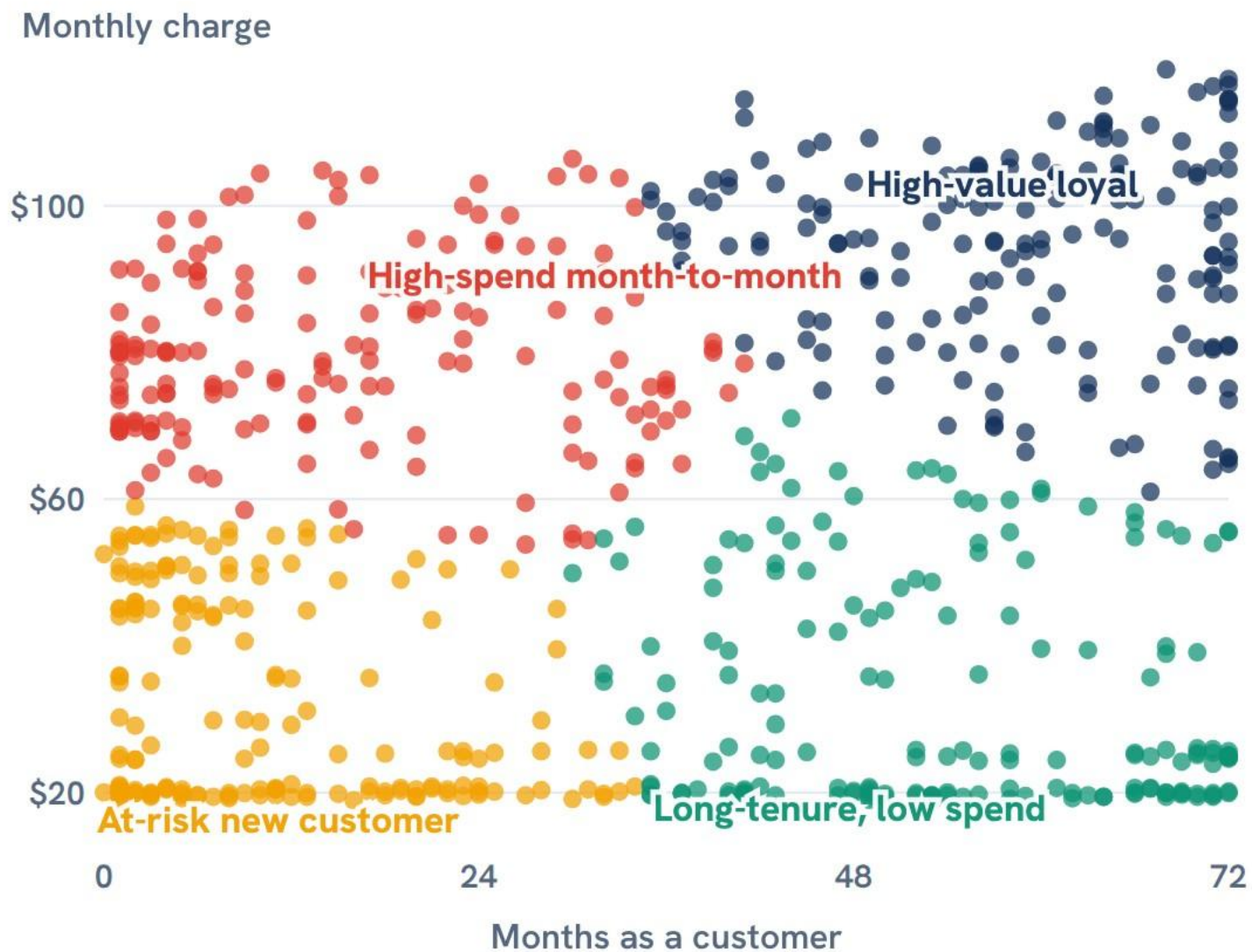
0 to 12 89%

13 to 36 77%

37 and up 39%

Of 214 churners on one or two year contracts, the model flagged 5.

Four groups, split by tenure and spend.



Why $k = 4$

Elbow: total spread inside groups



Silhouette: how well separated



Silhouette is highest at $k = 2$ (0.480). $k = 4$ is a local peak (0.472) and sits at the elbow. Two groups cannot split high from low spend within each tenure band.

Every score comes with its reasons.

● Pushes the risk up

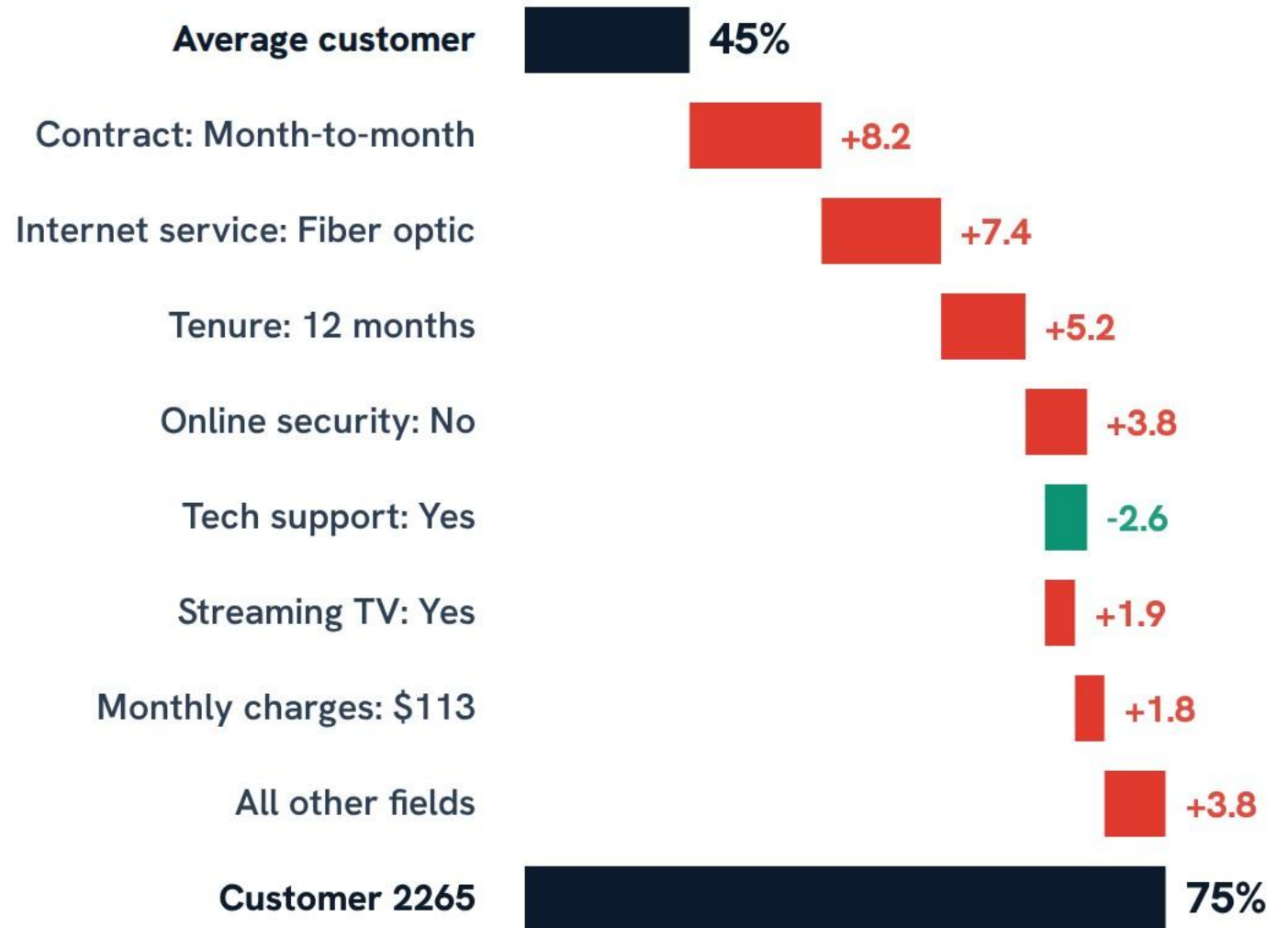
● Pushes the risk down

Bar length

Points of churn risk

SHAP splits the gap between this customer's score and the average score across the fields. The bars add up exactly.

Customer 2265: why the score is 75%



Customer 2265 scores 75%. Mostly because of a month-to-month contract and fiber optic internet.

Lifetime value is the monthly bill times the months we expect to keep them.

months left = $1 \div \text{score}$
capped at 60

Why $1 \div \text{score}$

In this dataset churn means leaving in the last month, so the score acts as a monthly chance. A constant chance of p gives an average life of $1 \div p$ months.

The catch

Balanced class weights inflate the scores. Use lifetime value to rank customers, not to forecast.

A \$65-a-month customer, at four risk scores

Score 75%
1.3 months expected

 **\$86**

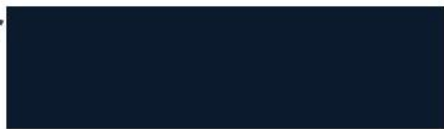
Score 45%
2.2 months expected

 **\$144**

Score 10%
10.0 months expected

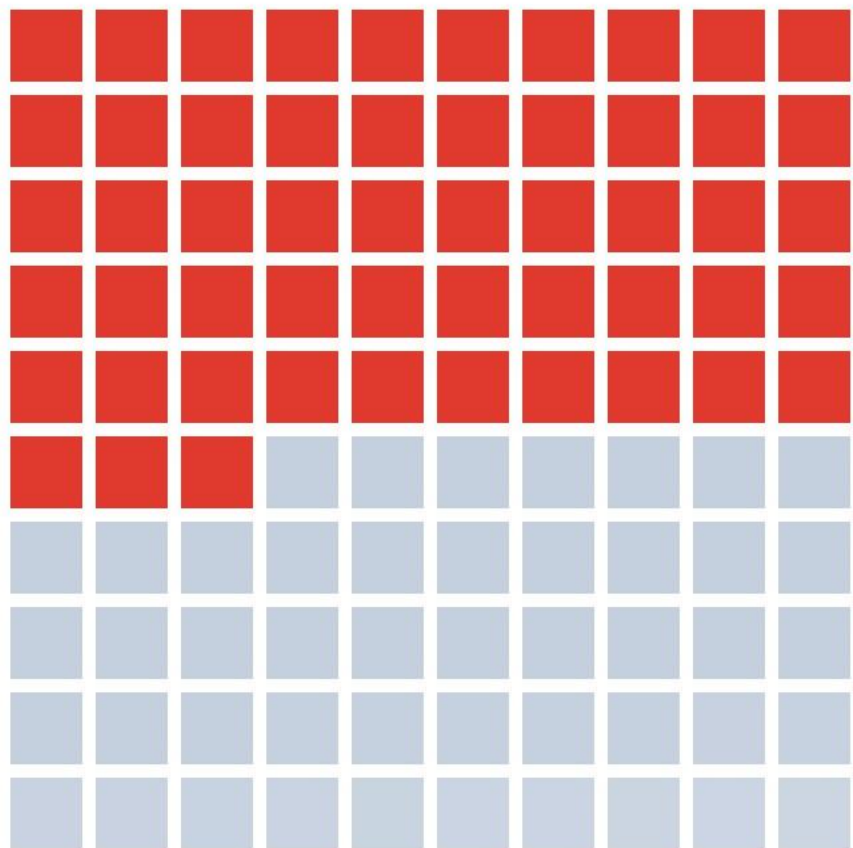
 **\$648**

Score 1.7% or lower
60 months, the cap

 **\$3,886**

100 customers the model flags as likely to churn

- 53 would have left
- 47 would have stayed. A false alarm costs one offer. A miss costs a customer.



Where it falls short, and what we would do next

1 A separate contract model

The current model flags 5 of 214 churners on one and two year contracts.

2 Calibrate the scores

Balanced weights inflate them. The medium tier scores 30 to 60%, and 26% of those customers churned.

3 A/B test the offers

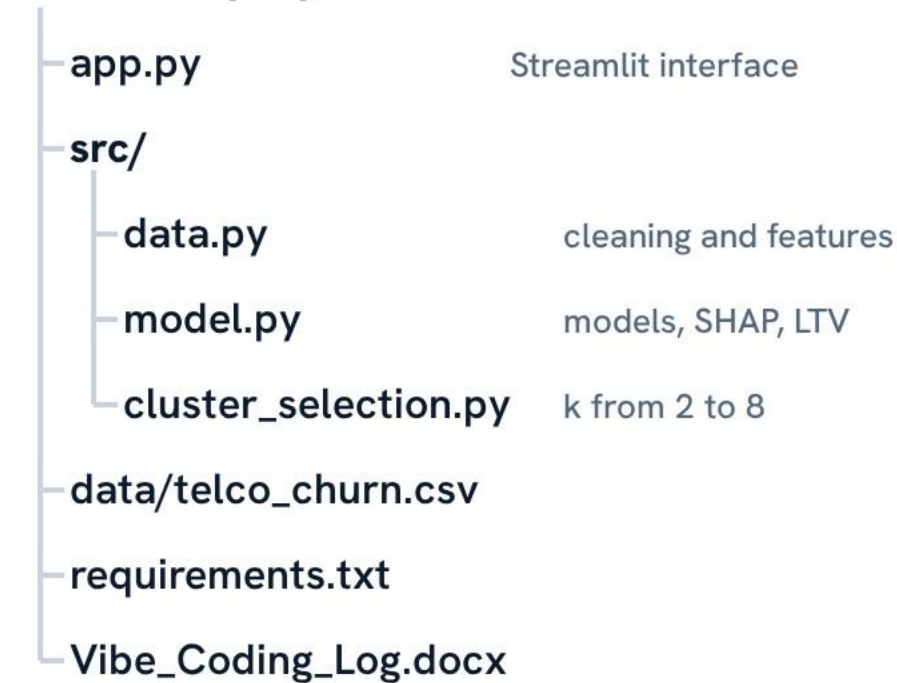
Replace our 20% save rate assumption with a measured one.

4 Add behavior data

The two models tie, so new signal such as support calls beats a fancier algorithm.

AI wrote the scaffolding. We made the decisions.

subscribeiq-capstone



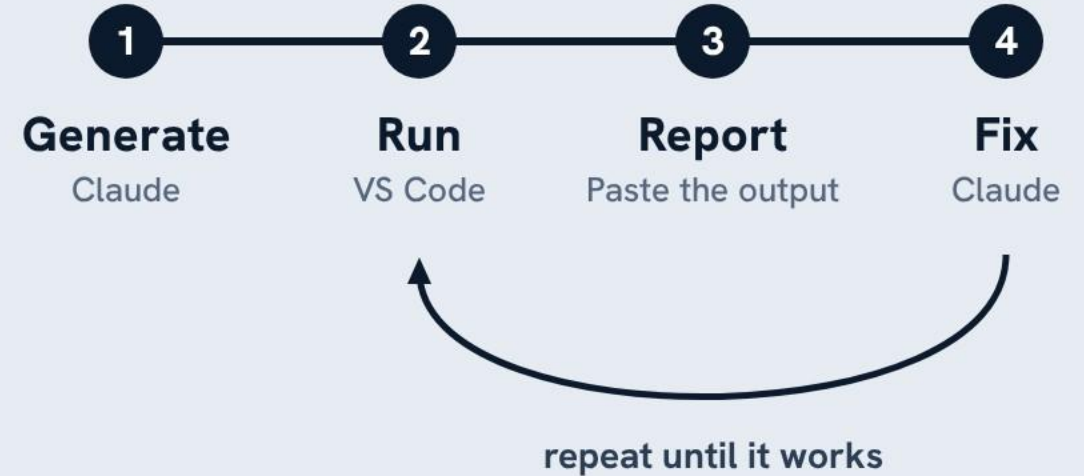
16 commits in 2 days



Oct 5: 5 commits

Oct 6: 11 commits

How the work ran

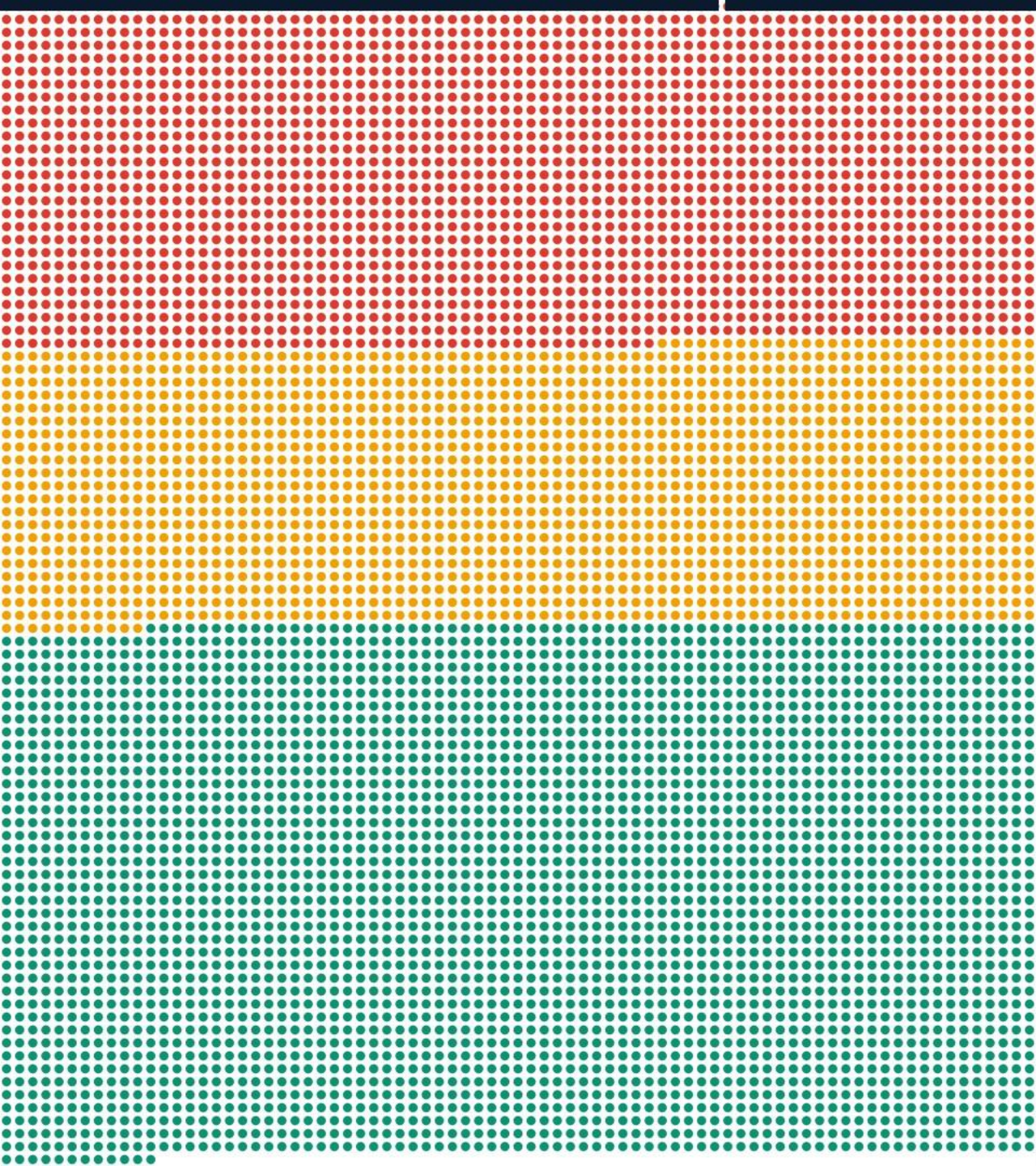


Constraint first. Algorithm, business rule and target file in one request.

Evidence for debugging. Paste the real error, not a description.

Small steps. Pipeline, app, SHAP, clusters, redesign, deploy.

Copilot and Streamlit. Copilot polished the UI in VS Code. Streamlit Community Cloud hosts the app.



A tool a manager can use,
on a pipeline we can
explain.

\$1.96M

a year of revenue sits in the
high-risk tier

0.847

cross-validated ROC-AUC

67%

of churners reached by
contacting the riskiest 30%

2.8x

return on offers, on our
stated assumptions

Questions

github.com/sumitks84/subscribeiq-capstone

Parameters, and why we chose them

Component	Setting	Why
Split	80/20, stratified, seed 42	Same 26.5% churn mix in train and test
Scaling	StandardScaler on 3 numeric fields	K-Means measures distance, so equal footing
Encoding	One-hot, unknown categories ignored	New categories score as zeros, no crash
Imbalance	class_weight balanced, churn counts 1.9x	Simpler to defend than SMOTE
Random forest	300 trees, max depth 8	Best of a 2 by 2 grid on ROC-AUC
Cross-validation	5-fold stratified, shuffled, seed 42	AUC 0.847 ± 0.009 , training data only
K-Means	k = 4, n_init 10, 3 scaled fields	Elbow at 4, silhouette 0.472
Risk tiers	Low under 0.3, High over 0.6	Business cut-offs, not the 0.5 decision rule
Lifetime value	Monthly charge $\times \min(1 \div \text{score}, 60)$	Expected life under a constant monthly chance
SHAP	TreeExplainer, 50-row background	Exact attributions that sum to the score